

Project Documentation Report

5-DOF Programmable Robotic Arm with Teach-Repeat Functionality

Microcontroller: Arduino Uno (ATmega328P)
Actuator Controller: PCA9685 16-Channel 12-bit PWM I2C Driver
Input Interface: 20kΩ Linear Potentiometers and Tactile Push Buttons

1. Abstract

This document outlines the hardware and software architecture of a 5-Degree of Freedom (5-DOF) robotic arm. The system features a "Teach and Repeat" operational mode, allowing users to manually guide the arm using potentiometers, save spatial coordinates (waypoints) into the microcontroller's RAM, and subsequently play back the recorded trajectory. The software integrates an Exponential Moving Average (EMA) filter to eliminate hardware noise and utilizes a Smoothstep (Hermite polynomial) interpolation algorithm to ensure fluid, jerk-free mechanical transitions during automated playback.

2. Hardware Bill of Materials (BOM)

- Arduino Uno (Main logic controller)
- PCA9685 Servo Driver Module (Generates stable 50Hz PWM signals)
- 5x Servo Motors (Standard high-torque servos for joints and gripper)
- 4x 20kΩ Linear Potentiometers (Manual joint angle controllers)
- 4x Tactile Push Buttons (Mode selection and gripper toggle)
- External Power Supply (5V-6V, minimum 3A capability dedicated to the PCA9685 module)

3. System Architecture & Pin Mapping

3.1 I2C Communication (Arduino to PCA9685)

Arduino Uno Pin	PCA9685 Pin	Description
A4	SDA	Serial Data Line
A5	SCL	Serial Clock Line (Operating at 100kHz)
5V / GND	VCC / GND	Logic power and common ground reference

3.2 Actuator Mapping (PCA9685 to Servos)

PCA9685 Channel	Joint Name	Functional Description
Channel 0	Base	Yaw rotation of the entire arm assembly.
Channel 1	Shoulder	Primary pitch axis for forward/backward lift.
Channel 2	Elbow	Secondary pitch axis. (Software inverted: 180 - target)
Channel 4	Wrist	Pitch axis for end-effector elevation.
Channel 5	Gripper	End-effector actuation (Open: 20° / Closed: 160°).

3.3 Input Interface Mapping

Component	Arduino Pin	Configuration	Function
Potentiometer 1	A0	Analog Input	Controls Base Servo (Channel 0).
Potentiometer 2	A1	Analog Input	Controls Shoulder Servo (Channel 1).
Potentiometer 3	A2	Analog Input	Controls Elbow Servo (Channel 2).
Potentiometer 4	A3	Analog Input	Controls Wrist Servo (Channel 4).
SAVE Button	D2	INPUT_PULLUP	Records current joint angles into the structural array.
PLAY Button	D3	INPUT_PULLUP	Toggles the continuous automated playback loop.
CLEAR Button	D4	INPUT_PULLUP	Flushes the RAM array and resets the sequence counter.
GRIPPER Button	D5	INPUT_PULLUP	Toggles the state of the end-effector (Open/Closed).

4. Software Design & Algorithms

4.1 Memory Optimization

To accommodate the limited 2KB SRAM of the Arduino Uno, the joint coordinates are stored using an array of structures containing uint8_t (1-byte) data types instead of standard 4-byte floats. This reduces the memory footprint per saved coordinate from 24 bytes to just 5 bytes, allowing a maximum of 100 positions to be stored safely without causing stack overflow.

4.2 Signal Conditioning (Noise Filtration)

Analog signals from potentiometers are subject to electrical jitter. The software applies a low-pass Exponential Moving Average (EMA) filter:

$$FilteredValue = (FilteredValue \times 0.80) + (RawADC \times 0.20)$$

Additionally, a deadband threshold of 0.8 degrees is implemented. The servos will not receive updated PWM pulses unless the manual input exceeds this threshold, completely eliminating static motor buzzing.

4.3 Trajectory Interpolation (Smoothstep)

During playback, directly commanding the servos from Point A to Point B causes abrupt jerks. The system uses a cubic Hermite interpolation formula (Ease-in, Ease-out) to calculate intermediate steps dynamically:

$$x = \text{progress (from 0.0 to 1.0)}$$

$$\text{EaseWeight} = x^2 \times (3 - 2x)$$

This mathematical curve ensures the motors accelerate smoothly from rest and decelerate gently as they approach the target coordinate.

5. Operating Instructions

1. **System Boot:** Upon powering the system, the arm defaults to an upright, neutral safe position (90° on all primary axes) and enters "Manual Mode".
2. **Teaching the Robot:** Rotate the 4 potentiometers to maneuver the arm into the desired physical position. If needed, press the Gripper Button (D5) to toggle the claw. Press the Save Button (D2) to record the frame. Repeat this process for multiple waypoints.
3. **Playback:** Press the Play Button (D3). The robot will disable manual inputs and autonomously navigate through all saved waypoints sequentially, looping indefinitely. Pressing Play again will abort the operation and return control to the potentiometers.
4. **Clearing Memory:** Pressing the Clear Button (D4) will permanently delete all recorded steps in the current session.

6. Complete Source Code (Arduino C++)

```
/**
 * 5-DOF Robotic Arm Controller with Teach & Repeat
 * Hardware: Arduino Uno, PCA9685, 4x 20k Potentiometers, 4x Push Buttons
 */
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);

// Servo timing configuration
#define SERVO_MIN 125
#define SERVO_MAX 575
#define SERVO_FREQ 50

// Operational constants
#define MAX_POSITIONS 100
#define DEBOUNCE_DELAY 50

// Trajectory and speed configuration for automated playback
#define STEPS_PER_MOVE 20
#define STEP_DELAY_MS 8

// Pin Definitions
const byte potPin[4] = {A0, A1, A2, A3};
const byte PIN_SAVE = 2;
const byte PIN_PLAY = 3;
const byte PIN_CLEAR = 4;
const byte PIN_GRIPPER = 5;

// Data structure optimized for SRAM utilization
struct JointPositions {
    uint8_t base;
    uint8_t shoulder;
    uint8_t elbow;
    uint8_t wrist;
};
```

```

    uint8_t gripper;
};

// Global variables for system state tracking
JointPositions savedMoves[MAX_POSITIONS];
int savedCount = 0;
int currentPlayIndex = 0;

float currentAngleBase      = 90.0;
float currentAngleShoulder = 90.0;
float currentAngleElbow     = 90.0;
float currentAngleWrist     = 90.0;
float filteredADC[4]        = {512.0, 512.0, 512.0, 512.0};
float liveGripAngle = 20.0;

bool gripperOpen = true;
bool isPlaying = false;

// Hardware debounce tracking variables
bool lastSaveState = HIGH;
bool lastPlayState = HIGH;
bool lastClearState = HIGH;
bool lastGripState = HIGH;

unsigned long lastSaveDebounce = 0;
unsigned long lastPlayDebounce = 0;
unsigned long lastClearDebounce = 0;
unsigned long lastGripDebounce = 0;

// Function Prototypes
void checkButtons();
void readManualInputs();
void executeSmoothPlaybackMove(JointPositions start, JointPositions target);
void setServo(byte ch, float angle);
float smoothstep(float edge0, float edge1, float x);

// =====
// INITIALIZATION
// =====
void setup() {
    Serial.begin(115200);

    // Initialize I2C with stable 100kHz clock speed
    Wire.begin();
    Wire.setClock(100000);

    // Initialize PCA9685 driver
    pca.begin();
    pca.setPWMPFreq(SERVO_FREQ);
    delay(500);

    // Calibrate joints to neutral safe positions
    setServo(0, currentAngleBase);
    setServo(1, currentAngleShoulder);
    setServo(2, currentAngleElbow);
    setServo(4, currentAngleWrist);
    setServo(5, liveGripAngle);

    // Configure input pins utilizing internal pull-up resistors
    pinMode(PIN_SAVE, INPUT_PULLUP);
    pinMode(PIN_PLAY, INPUT_PULLUP);
    pinMode(PIN_CLEAR, INPUT_PULLUP);
    pinMode(PIN_GRIPPER, INPUT_PULLUP);

    Serial.println(F("System Initialization Complete. Manual Mode Active.));
}

// =====
// MAIN EXECUTION LOOP
// =====
void loop() {
    checkButtons();

    if (isPlaying) {

```

```

    if (savedCount == 0) {
        isPlaying = false;
        Serial.println(F("Error: Sequence memory empty."));
    } else {
        Serial.print(F("Executing Sequence: "));
        Serial.print(currentPlayIndex + 1);
        Serial.print(F(" / "));
        Serial.println(savedCount);

        // Package current local variables into struct format
        JointPositions startState = {
            (uint8_t)currentAngleBase, (uint8_t)currentAngleShoulder,
            (uint8_t)currentAngleElbow, (uint8_t)currentAngleWrist,
            (uint8_t)liveGripAngle
        };

        JointPositions targetState = savedMoves[currentPlayIndex];

        // Initiate trajectory execution
        executeSmoothPlaybackMove(startState, targetState);

        // Increment array index or loop back to origin
        currentPlayIndex++;
        if (currentPlayIndex >= savedCount) {
            currentPlayIndex = 0;
        }
    } else {
        // Process manual inputs if playback is disabled
        readManualInputs();
        delay(10);
    }
}

// =====
// TACTILE SWITCH INTERRUPT HANDLING (POLLING)
// =====
void checkButtons() {
    unsigned long currentTime = millis();

    // --- SAVE BUTTON PROCESSING ---
    bool readingSave = digitalRead(PIN_SAVE);
    if (readingSave != lastSaveState) lastSaveDebounce = currentTime;
    if ((currentTime - lastSaveDebounce) > DEBOUNCE_DELAY) {
        if (readingSave == LOW && !isPlaying) {
            if (savedCount < MAX_POSITIONS) {
                savedMoves[savedCount] = {
                    (uint8_t)currentAngleBase, (uint8_t)currentAngleShoulder,
                    (uint8_t)currentAngleElbow, (uint8_t)currentAngleWrist,
                    (uint8_t)liveGripAngle
                };
                savedCount++;
                Serial.print(F("Waypoint Saved. Total: "));
                Serial.println(savedCount);
            } else {
                Serial.println(F("Warning: Memory capacity reached."));
            }
            lastSaveDebounce = currentTime + 400; // Hardlock to prevent multiple triggers
        }
    }
    lastSaveState = readingSave;

    // --- PLAY BUTTON PROCESSING ---
    bool readingPlay = digitalRead(PIN_PLAY);
    if (readingPlay != lastPlayState) lastPlayDebounce = currentTime;
    if ((currentTime - lastPlayDebounce) > DEBOUNCE_DELAY) {
        if (readingPlay == LOW) {
            isPlaying = !isPlaying;
            currentPlayIndex = 0; // Reset index upon activation
            Serial.println(isPlaying ? F("Mode: AUTONOMOUS PLAYBACK") : F("Mode: MANUAL CONTROL"));
            lastPlayDebounce = currentTime + 400;
        }
    }
}

```

```

lastPlayState = readingPlay;

// --- CLEAR BUTTON PROCESSING ---
bool readingClear = digitalRead(PIN_CLEAR);
if (readingClear != lastClearState) lastClearDebounce = currentTime;
if ((currentTime - lastClearDebounce) > DEBOUNCE_DELAY) {
    if (readingClear == LOW) {
        savedCount = 0;
        currentPlayIndex = 0;
        isPlaying = false;
        Serial.println(F("System Memory Flushed."));
        lastClearDebounce = currentTime + 400;
    }
}
lastClearState = readingClear;

// --- GRIPPER BUTTON PROCESSING ---
bool readingGrip = digitalRead(PIN_GRIPPER);
if (readingGrip != lastGripState) lastGripDebounce = currentTime;
if ((currentTime - lastGripDebounce) > DEBOUNCE_DELAY) {
    if (readingGrip == LOW) {
        gripperOpen = !gripperOpen;
        liveGripAngle = gripperOpen ? 20.0 : 160.0;
        setServo(5, liveGripAngle);
        Serial.println(gripperOpen ? F("End-Effector: DEPLOYED (OPEN)") : F("End-Effector: SECURED (CLOSED)"));
        lastGripDebounce = currentTime + 400;
    }
}
lastGripState = readingGrip;
}

// =====
// ANALOG SIGNAL CONDITIONING & ACTUATION
// =====
void readManualInputs() {
    // --- AXIS 1: BASE (CHANNEL 0) ---
    int rawADC0 = analogRead(potPin[0]);
    filteredADC[0] = filteredADC[0] * 0.80 + rawADC0 * 0.20;
    float targetBase = map(filteredADC[0], 0, 1023, 0, 180);
    if(abs(targetBase - currentAngleBase) > 0.8) {
        currentAngleBase += (targetBase - currentAngleBase) * 0.25;
        setServo(0, currentAngleBase);
    }

    // --- AXIS 2: SHOULDER (CHANNEL 1) ---
    int rawADC1 = analogRead(potPin[1]);
    filteredADC[1] = filteredADC[1] * 0.80 + rawADC1 * 0.20;
    float targetShoulder = map(filteredADC[1], 0, 1023, 0, 180);
    if(abs(targetShoulder - currentAngleShoulder) > 0.8) {
        currentAngleShoulder += (targetShoulder - currentAngleShoulder) * 0.25;
        setServo(1, currentAngleShoulder);
    }

    // --- AXIS 3: ELBOW (CHANNEL 2) ---
    int rawADC2 = analogRead(potPin[2]);
    filteredADC[2] = filteredADC[2] * 0.80 + rawADC2 * 0.20;
    float targetElbow = map(filteredADC[2], 0, 1023, 0, 180);
    targetElbow = 180.0 - targetElbow; // Logical inversion for hardware alignment
    if(abs(targetElbow - currentAngleElbow) > 0.8) {
        currentAngleElbow += (targetElbow - currentAngleElbow) * 0.25;
        setServo(2, currentAngleElbow);
    }

    // --- AXIS 4: WRIST (CHANNEL 4) ---
    int rawADC3 = analogRead(potPin[3]);
    filteredADC[3] = filteredADC[3] * 0.80 + rawADC3 * 0.20;
    float targetWrist = map(filteredADC[3], 0, 1023, 0, 180);
    if(abs(targetWrist - currentAngleWrist) > 0.8) {
        currentAngleWrist += (targetWrist - currentAngleWrist) * 0.25;
        setServo(4, currentAngleWrist);
    }

    // Ensure gripper state is maintained during manual tracking

```

```

    setServo(5, liveGripAngle);

    // Output telemetry data via Serial interface
    Serial.print("Base: ");      Serial.print(currentAngleBase, 1);      Serial.print(" | ");
    Serial.print("Shoulder: ");  Serial.print(currentAngleShoulder, 1);  Serial.print(" | ");
    Serial.print("Elbow: ");     Serial.print(currentAngleElbow, 1);     Serial.print(" | ");
    Serial.print("Wrist: ");     Serial.print(currentAngleWrist, 1);     Serial.print(" | ");
    Serial.print("Gripper: ");   Serial.print(liveGripAngle, 1);         Serial.println();
}

// =====
// CUBIC TRAJECTORY GENERATION (PLAYBACK)
// =====
void executeSmoothPlaybackMove(JointPositions start, JointPositions target) {
    for (int step = 0; step <= STEPS_PER_MOVE; step++) {
        // Calculate normalized progress (0.0 to 1.0)
        float progress = (float)step / (float)STEPS_PER_MOVE;

        // Apply Hermite polynomial mapping for non-linear acceleration
        float easeWeight = smoothstep(0.0, 1.0, progress);

        // Compute interpolation for all axes simultaneously
        float interBase      = (float)start.base      + ((float)target.base      -
(float)start.base)      * easeWeight;
        float interShoulder = (float)start.shoulder + ((float)target.shoulder -
(float)start.shoulder) * easeWeight;
        float interElbow    = (float)start.elbow    + ((float)target.elbow    -
(float)start.elbow)    * easeWeight;
        float interWrist    = (float)start.wrist    + ((float)target.wrist    -
(float)start.wrist)    * easeWeight;
        float interGripper  = (float)start.gripper  + ((float)target.gripper  -
(float)start.gripper)  * easeWeight;

        // Transmit computed targets to hardware driver
        setServo(0, interBase);
        setServo(1, interShoulder);
        setServo(2, interElbow);
        setServo(4, interWrist);
        setServo(5, interGripper);

        // Synchronize global variables to maintain accurate location tracking
        currentAngleBase      = interBase;
        currentAngleShoulder = interShoulder;
        currentAngleElbow    = interElbow;
        currentAngleWrist    = interWrist;
        liveGripAngle        = interGripper;

        delay(STEP_DELAY_MS);

        // Safety Interrupt: Allows user to abort operation mid-trajectory
        bool readingPlay = digitalRead(PIN_PLAY);
        if (readingPlay == LOW && lastPlayState == HIGH) {
            isPlaying = false;
            Serial.println(F("Operation Aborted: User override."));
            break;
        }
    }
}

// =====
// HARDWARE ABSTRACTION LAYER (HAL)
// =====
void setServo(byte ch, float angle) {
    angle = constrain(angle, 0.0, 180.0);
    int pulse = map((int)angle, 0, 180, SERVO_MIN, SERVO_MAX);
    pca.setPWM(ch, 0, pulse);
}

// Mathematics utility for ease-in ease-out curve generation
float smoothstep(float edge0, float edge1, float x) {
    x = constrain((x - edge0) / (edge1 - edge0), 0.0, 1.0);
    return x * x * (3 - 2 * x);
}

```